CS308 — Software Engineering

Aktour ViaBalkan

System Design Document

UML Class · Use Case · Sequence Diagrams

Project Team

Furkan Duman · Dogukan Yurtturk · Eren Çapar · Can Aysen

International University of Sarajevo

April 2026 · Version 1.0

Contents

1 Introduction **2**
1.1 Project Scope . . . 2
1.2 Actors . . . 2
1.3 Document Structure . . . 2

2 Class Diagram **3**
2.1 How Classes Were Identified . . . 3
2.1.1 User (abstract) . . . 3
2.1.2 Admin . . . 3
2.1.3 Guide . . . 3
2.1.4 Tour . . . 3
2.1.5 Route . . . 3
2.1.6 Waypoint . . . 4
2.1.7 Vehicle . . . 4
2.1.8 Toll . . . 4
2.1.9 Expense . . . 5
2.2 Class Relationships . . . 5
2.3 Class Diagram . . . 6
2.4 Class Summary . . . 6

3 Use Case Diagram **8**
3.1 Identification of Use Cases . . . 8
3.2 Use Case Table . . . 8
3.3 Use Case Diagram . . . 10

4 Sequence Diagram — Create New Tour **11**
4.1 Why This Scenario? . . . 11
4.2 Participants (Lifelines) . . . 11
4.3 Step-by-Step Flow . . . 11
4.4 Sequence Diagram . . . 13

5 Key Design Decisions **14**
5.1 Abstract User Class . . . 14
5.2 Waypoint as Composition of Route . . . 14
5.3 Guide Languages and Countries . . . 14
5.4 Route as a Separate Class . . . 14
5.5 Expense as Composition of Tour . . . 14
5.6 Vehicle as an Aggregated Resource . . . 14
5.7 Toll as a Shared Aggregated Entity . . . 15
5.8 REST API as Sole Communication Channel . . . 15

1 Introduction

> *Aktour ViaBalkan is a desktop application that automates tour planning, expense tracking, and financial reporting for tour operators in the Balkan region. Stack: JavaFX (UI), Spring Boot (REST API backend), MySQL (database), Ubuntu Server.*

1.1 Project Scope

The system addresses the problem of tour companies relying on manual calculations and generic management tools that lack region-specific data for the Balkans. Aktour ViaBalkan provides:

- Automated expense tracking with receipt upload.
- Tour creation wizard with dynamic cost calculation (fuel, tolls, guide fees).
- Guide profile management including languages spoken and countries of operation.
- Flexible route waypoints — optional stops that may or may not be included in the paid fare.
- Role-based access control separating Admin and Guide views.
- Real-time financial dashboards for tour profitability analysis.

1.2 Actors

Actor	Role	Access Level
Admin (Owner)	Creates and manages tours, guides, vehicles, and routes. Views financial dashboards and generates reports.	Full access
Tour Guide	Views assigned tours, uploads receipts, updates tour info, manages own profile.	Limited — own data only

Table 1. *System Actors*

1.3 Document Structure

This document contains three UML diagrams along with explanations of design decisions:

- **Section 2 — Class Diagram:** static structure of all domain entities and their relationships.
- **Section 3 — Use Case Diagram:** functional scope from each actor's perspective.
- **Section 4 — Sequence Diagram:** step-by-step interaction flow for "Create New Tour".
- **Section 5 — Key Design Decisions:** rationale behind the main architectural choices.

2 Class Diagram

2.1 How Classes Were Identified

Classes were identified by analysing the entities, responsibilities, and data requirements described in the SRS and Project Proposal. The following reasoning was applied for each class:

2.1.1 User (abstract)

Both Admin and Guide share identity attributes (`username`, `email`, `passwordHash`, `role`). An abstract base class avoids duplicating authentication logic. The `role` field drives RBAC at the Spring Boot API layer.

2.1.2 Admin

Extends `User`. The Admin is the system owner — responsible for creating tours, managing resources (guides, vehicles, routes), and viewing financial dashboards. No additional attributes beyond `User`; all behaviour is captured in methods.

2.1.3 Guide

Extends `User`. Guides carry domain-specific professional data:

- `languages : List<String>` — languages the guide speaks (e.g. Bosnian, English, Turkish, German). Used when matching guides to tour groups.
- `countries : List<String>` — countries in which the guide is licensed and experienced (e.g. Bosnia, Croatia, Slovenia). Essential for Balkan multi-country tours.
- `skills : List<String>` — specialisations such as hiking, city tours, historical sites, cuisine.
- `experience : int` — years of active guiding. Used in search filters and cost calculation.
- `rating : float` — aggregated customer satisfaction score (0.0 – 5.0).

2.1.4 Tour

The central domain entity. Aggregates start/end dates, computed `totalCost`, and links to a Route, a Vehicle, and a list of Expenses. The `calculateCost()` method reads toll and fuel data from the associated Route and Vehicle. `hotelName` stores the name of the accommodation used during the tour as a simple string field.

2.1.5 Route

Encapsulates Balkan-specific geographical data: start/end cities, total distance, and the list of tolls encountered along that leg. Required by REQ-4 (dynamic cost calculation). A Route owns zero or more Waypoints via composition — when a Route is deleted, its Waypoints are also removed — and aggregates zero or more Toll records. Toll records are *shared* infrastructure entities that are not removed when a Route is deleted.

- `addWaypoint()` allows the guide or admin to register an extra stop during or before the tour.
- The old scalar `tollCost` field has been replaced by an aggregated list of Toll objects,

so per-vehicle-category pricing is applied automatically.

2.1.6 Waypoint

> *Business rule: A customer who paid for Sarajevo → Mostar is only billed for that route. If the guide makes an additional stop (e.g. Konjic), it becomes a Waypoint with `additionalCost > 0` and `isPaid = false` until the customer settles the extra charge.*

Key fields:

- `city / country` — location of the optional stop.
- `additionalCost : float` — 0 if the stop is complimentary, > 0 if it incurs an extra charge.
- `isPaid : boolean` — tracks whether the customer has settled the extra cost.
- `calcExtraCost()` — returns `additionalCost` if `isPaid` is false, otherwise 0.

2.1.7 Vehicle

A detailed record of each physical vehicle in the fleet. The expanded attribute set was introduced to support:

- `brand / model / year / color` — distinguishes between visually similar vehicles (e.g. two white Sprinter minibuses with different registration years).
- `plateNumber` — unique legal identifier; used on official tour documents and for insurance purposes.
- `capacity` — maximum passenger count; drives vehicle selection in the Tour wizard.
- `fuelType / avgFuelConsumption` — fuel type (diesel, petrol, LPG) and consumption in litres per 100 km. Used by `Tour.calculateCost()` to derive the fuel cost for a given route distance.
- `currentMileage` — odometer reading; updated after each tour.
- `lastMinorService` — date of the most recent scheduled service; triggers a maintenance alert when the next service interval is due.
- `serviceHistory : List<String>` — log of all past service events with dates and descriptions (e.g. "2025-11-14: oil change + brake pads").

The relationship between Tour and Vehicle is **aggregation** (◇, hollow diamond): a Vehicle is a long-lived company asset managed on its own maintenance schedule and can be reused across many Tours.

2.1.8 Toll

> *Real-world example — Sarajevo → Mostar route:*
>
> - *Tarčin highway toll: Cat I (car) = 3.50 KM | Cat II (minibus) = 7.00 KM | Cat III (coach) = 10.50 KM*
> - *Buna toll: Cat I (car) = 5.00 KM | Cat II (minibus) = 9.00 KM | Cat III (coach) = 13.00 KM*

Toll booths in the Balkans apply three tariff categories based on vehicle size:

- `name / location` — human-readable identifier and GPS-friendly location string.
- `costCat1 : float` — fee for Category I vehicles (cars, small SUVs).
- `costCat2 : float` — fee for Category II vehicles (minibuses, vans $\leq 3.5\,\mathrm{t}$).
- `costCat3 : float` — fee for Category III vehicles (coaches, buses $> 3.5\,\mathrm{t}$).
- `getCostFor(vehicleType : String) : float` — returns the correct fee at runtime based on the assigned Vehicle's type.

The Route ◇→ Toll relationship is **aggregation**: the toll booth physically exists regardless of any particular route or tour, and multiple Route records can reference the same Toll.

2.1.9 Expense

Tracks individual cost line-items per tour (fuel receipts, toll receipts, guide fees, miscellaneous). Composed inside Tour — lifecycle is tied to the Tour. Guides upload receipts stored at `receiptPath`.

2.2 Class Relationships

Relationship	Type	Multiplicity	Description
Admin / Guide → User	Inheritance	—	Both subclasses inherit authentication logic from User.
Admin → Tour	Association	1 → 0..∗	An Admin creates and manages many Tours.
Guide → Tour	Association	1 → 0..∗	A Guide is assigned to many Tours over time.
Tour → Route	Association	1 → 1..∗	A Tour follows one or more route legs.
Route ♦→ Waypoint	Composition ♦	1 → 0..∗	A Route owns Waypoints. Removed with Route.
Tour ♦→ Expense	Composition ♦	1 → 0..∗	A Tour owns its Expense records. Removed with Tour.
Tour ◇→ Vehicle	Aggregation ◇	0..∗ → 1	Many Tours can use the same Vehicle independently.
Route ◇→ Toll	Aggregation ◇	1 → 0..∗	A Route references Toll records. Tolls are shared entities.

Table 2. *Class Relationships Summary*

2.3 Class Diagram

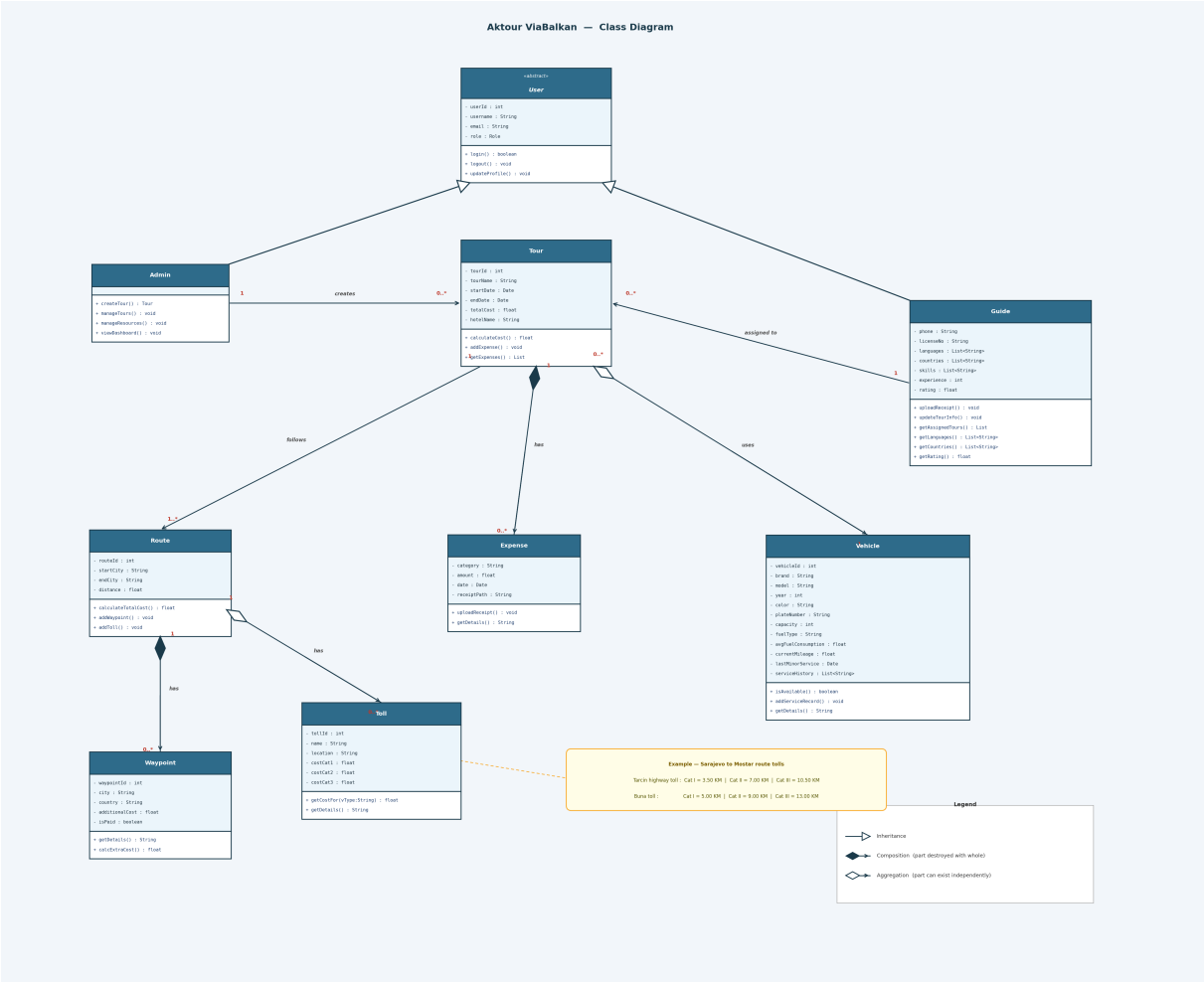

Figure 1. *Aktour ViaBalkan — Class Diagram*

2.4 Class Summary

Class	Attributes	Methods
User	userId, username, email, role	login(), logout(), updateProfile()
Admin	(inherits User)	createTour(), manageTours(), manageResources(), viewDashboard()
Guide	phone, licenseNo, languages, countries, skills, experience, rating	uploadReceipt(), updateTourInfo(), getAssignedTours(), getLanguages(), getCountries(), getRating()

Class	Attributes	Methods
Tour	tourId, tourName, startDate, endDate, totalCost, hotelName	calculateCost(), addExpense(), getExpenses()
Route	routeId, startCity, endCity, distance	calculateTotalCost(), addWaypoint()
Waypoint	waypointId, city, country, additionalCost, isPaid	getDetails(), calcExtraCost()
Vehicle	vehicleId, brand, model, year, color, plateNumber, capacity, fuelType, avgFuelConsumption, currentMileage, lastMinorService, serviceHistory	isAvailable(), addServiceRecord(), getDetails()
Toll	tollId, name, location, costCat1, costCat2, costCat3	getCostFor(vType), getDetails()
Expense	category, amount, date, receiptPath	uploadReceipt(), getDetails()

3 Use Case Diagram

3.1 Identification of Use Cases

Use cases were derived by mapping each SRS system feature to observable interactions between actors and the system:

- Feature 3.1 (Secure Login / RBAC) → Login, Logout for both actors.
- Feature 3.2 (Admin Dashboard) → View Dashboard, Generate Report.
- Feature 3.3 (Tour Creation Wizard) → Create New Tour ⟨⟨includes⟩⟩ Calculate Tour Cost.
- Feature 3.4 (Expense Management) → Upload Expense Receipt (Guide), View Expenses (both).
- Feature 3.5 (Guide / Vehicle / Route Management) → Manage Guides, Manage Vehicles, Manage Routes, Manage Waypoints.
- Guide profile features (SRS §2.3) → Update Profile, View Assigned Tours, Update Tour Info.

3.2 Use Case Table

Actor	Use Case	Description
Admin	Login / Logout	Authenticate with credentials; end session securely.
Admin	View Dashboard	See financial charts of tour revenue, costs, and profitability.
Admin	Create New Tour	Step-by-step wizard: select route, assign guide and vehicle, get auto-calculated cost.
Admin	Manage Guides	Add, update, or remove guide profiles (incl. languages, countries, skills).
Admin	Manage Vehicles	Add, update, or remove vehicle records.
Admin	Manage Routes	Define routes with start/end cities, distance, and toll data.
Admin	Manage Waypoints	Add optional stops to a route; set additional cost and paid status.
Admin	View All Expenses	Browse every expense entry across all tours and guides.
Admin	Generate Report	Export a profitability report for a tour or date range.

Actor	Use Case	Description
Guide	Login / Logout	Authenticate with credentials; end session securely.
Guide	View Assigned Tours	See the list of tours currently or previously assigned to the guide.
Guide	Update Tour Info	Edit operational details (status, notes) of an ongoing tour.
Guide	Upload Expense Receipt	Attach a photo/file receipt and log the expense category and amount.
Guide	View Own Expenses	Review all expense entries the guide has submitted.
Guide	Update Profile	Edit languages, countries, skills, contact info.
Both	Calculate Tour Cost	⟨⟨include⟩⟩ — triggered from Create Tour; auto-calculates fuel, tolls, guide fees.

3.3 Use Case Diagram

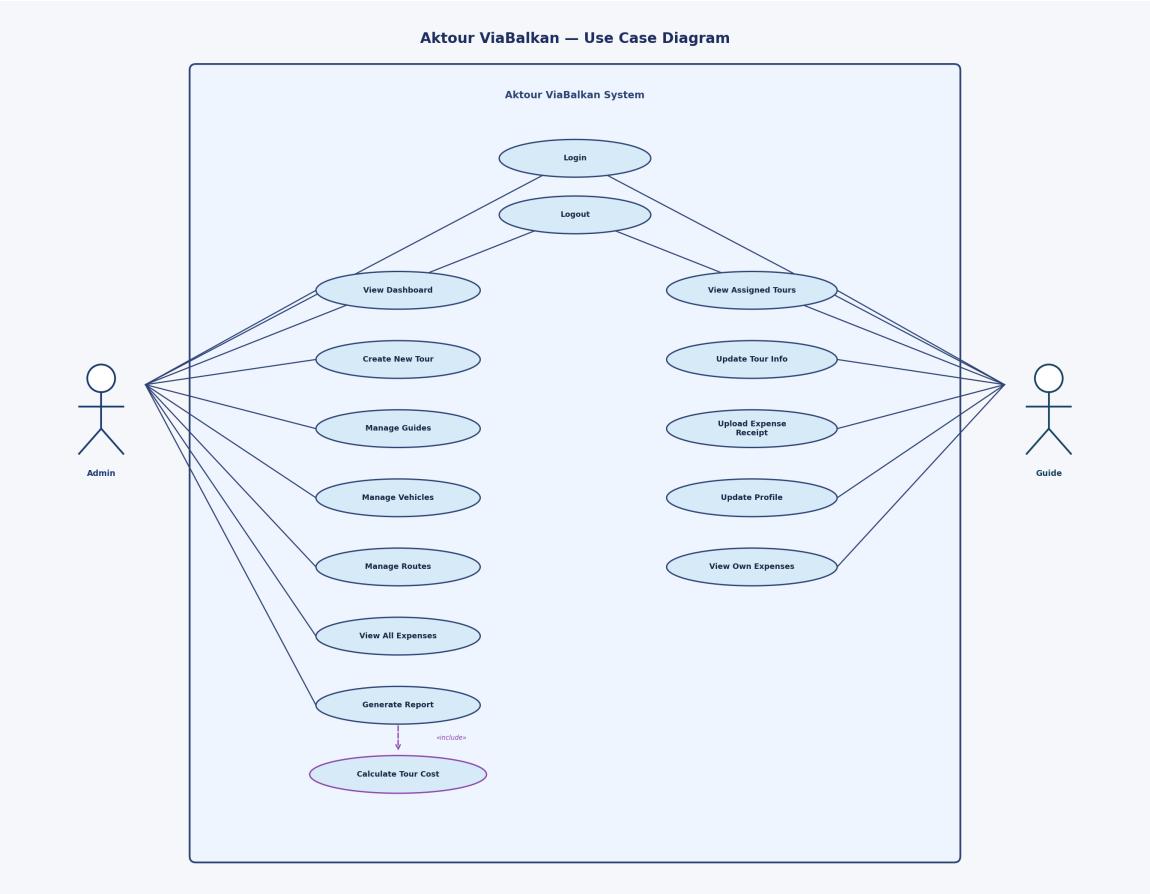

Figure 2. *Aktour ViaBalkan — Use Case Diagram*

4 Sequence Diagram — Create New Tour

4.1 Why This Scenario?

The "Create New Tour" flow was selected because it is the highest-priority feature (REQ-4) and exercises all four system participants: the Admin actor, the JavaFX desktop UI, the Spring Boot REST API, and the MySQL database. It demonstrates RBAC-gated access, real-time cost calculation from Balkan-specific route data, and data persistence.

4.2 Participants (Lifelines)

Participant	Role in Flow
Admin	Initiates the wizard, selects route/guide/vehicle, reviews the cost estimate, and confirms creation.
JavaFX UI	Manages form state, sends HTTP requests to the API via HttpClient, and renders responses and error messages.
Spring Boot API	Validates incoming requests, applies business logic (cost calculation using Route and Vehicle data), and orchestrates DB operations.
MySQL DB	Persistent store for tours, routes, guides, vehicles, waypoints, and expenses.

Table 5. *Sequence Diagram Participants*

4.3 Step-by-Step Flow

#	From → To	Message	Notes
1	Admin → UI	openTourWizard()	Admin clicks "New Tour" button.
2	UI → API	GET /routes, /guides, /vehicles	Fetch dropdown data for the form.
3	API → DB	SELECT routes, guides, vehicles	Query available resources.
4	DB → API	Result sets	Returns rows to API.
5	API → UI	JSON response	UI populates dropdowns.
6	Admin → UI	selectRoute(), assignGuide(), assignVehicle()	Admin fills in the wizard form.

#	From → To	Message	Notes
7	UI → API	GET /calculateCost(routeId, vehicleId)	Request live cost estimate.
8	API → DB	SELECT toll, distance, fuelEfficiency	Retrieve route and vehicle data.
9	DB → API	Cost data	Returns numbers for calculation.
10	API → UI	costEstimate : float	API returns calculated total cost.
11	Admin → UI	confirmCreate()	Admin approves and submits form.
12	UI → API	POST /tours {tourData, guideId, vehicleId}	Send full tour payload.
13	API → DB	INSERT INTO tours VALUES (...)	Persist the new tour record.
14	DB → API	tourId : int	DB returns generated primary key.
15	API → UI	201 Created { tourId }	Success response with new ID.
16	UI → Admin	showSuccess("Tour Created!")	UI confirms creation to Admin.

4.4 Sequence Diagram

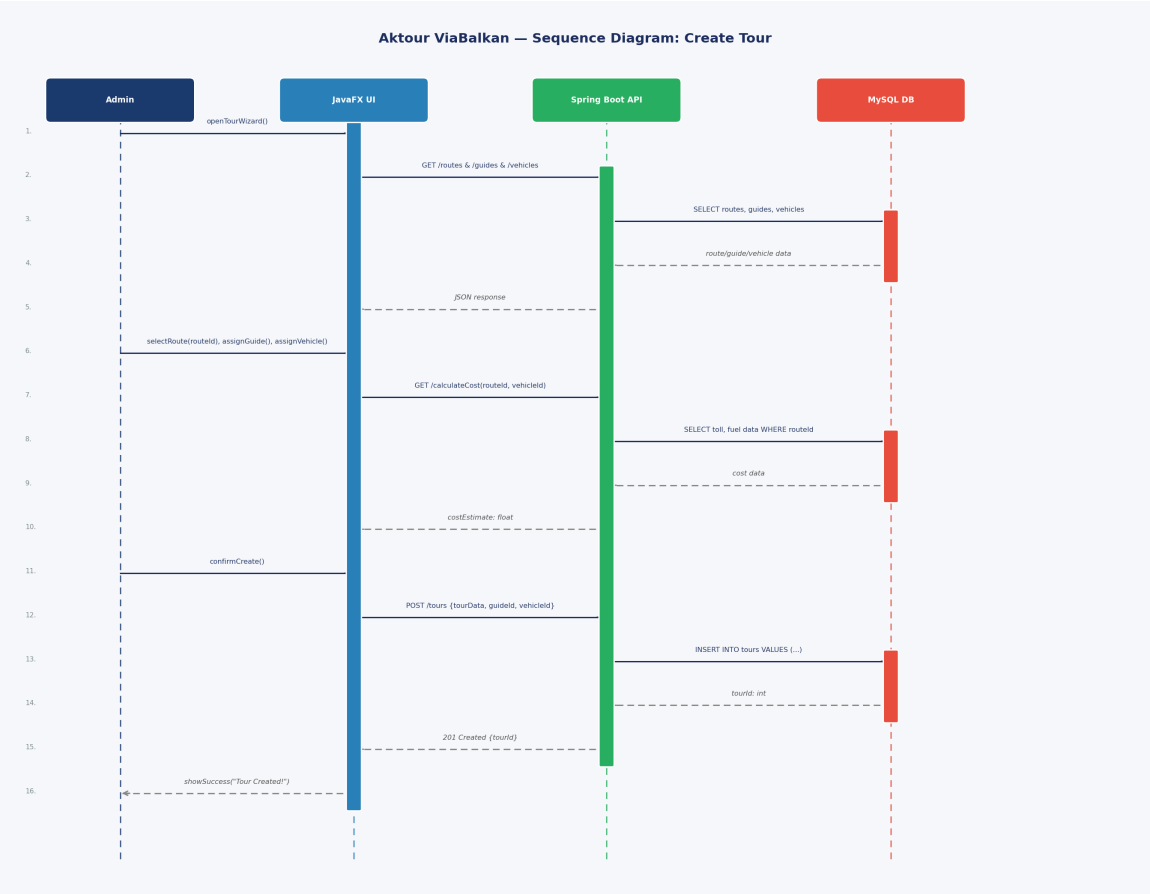

Figure 3. *Sequence Diagram — Create New Tour*

5 Key Design Decisions

5.1 Abstract User Class

Placing shared authentication fields (`username`, `email`, `passwordHash`, `role`) in an abstract `User` class eliminates duplication. RBAC is enforced at the Spring Boot API layer by inspecting the `role` field on every request — Guides receive HTTP 403 when they attempt Admin-only endpoints.

5.2 Waypoint as Composition of Route

> *Key scenario: A customer pays for Sarajevo → Mostar. The guide adds an optional stop in Konjic. The Waypoint record stores `additionalCost = 25.00` and `isPaid = false`. The system flags it as an outstanding charge until the customer settles it.*

Waypoints are composed inside Route (not Tour) because a stop is geographically tied to the route segment, not to a specific booking. The same Route can be reused across many Tours while carrying its known optional stops. If the Route is removed, its Waypoints are removed too (composition ownership).

5.3 Guide Languages and Countries

Storing `languages` and `countries` as `List<String>` (rather than foreign keys to lookup tables) keeps the data model simple for the initial version. Both fields support guide-to-tour matching — an Admin can filter guides by country or language when assigning to an international tour group. In a future version, these could be normalised into separate Language and Country entities.

5.4 Route as a Separate Class

Balkan-specific geographical data (toll costs, distances between cities) changes independently of tour bookings. Keeping Route separate allows: (1) reuse of a Sarajevo–Mostar route record across many tours, (2) bulk updates when toll rates change, and (3) attachment of Waypoints and Toll records that apply to all tours on that route.

5.5 Expense as Composition of Tour

An Expense record has no standalone meaning without its parent Tour. Using composition ensures that deleting a Tour also removes all associated Expense records, preventing orphaned data in the MySQL database.

5.6 Vehicle as an Aggregated Resource

The Tour ◇→ Vehicle link uses **aggregation** (hollow diamond ◇) rather than composition. The key distinction:

- A Vehicle is a long-lived company asset managed on its own maintenance schedule — it is not owned or destroyed by any single Tour.
- The same Vehicle can be reused across many Tours at different time intervals; the system checks `isAvailable()` to prevent double-booking.
- Vehicle attributes such as `currentMileage` and `serviceHistory` persist across all tours and must be updated incrementally, not re-created per booking.

The expanded 12-field attribute set also supports fleet management: operators can filter by `fuelType` when fuel costs change, or sort by `lastMinorService` to schedule maintenance proactively.

5.7 Toll as a Shared Aggregated Entity

> *Design rationale: The Sarajevo–Mostar route passes through both the Tarčin and Buna toll booths. Both toll records are shared infrastructure — they also appear on other routes (e.g. Sarajevo–Mostar–Dubrovnik). Deleting one route must not delete a toll that other routes depend on.*

Key decisions for the Toll class:

- Toll is a separate class (not a scalar field on Route) because toll fees vary by vehicle category. A single `costCat1` / `costCat2` / `costCat3` model covers cars (Cat I), minibuses/vans (Cat II), and coaches/buses (Cat III) without any conditional logic in calling code.
- `getCostFor(vehicleType)` encapsulates the lookup so that `Tour.calculateCost()` simply iterates `route.getTolls()` and calls `getCostFor(vehicle.type)` on each — no switch-case logic outside the Toll class.
- Route ◇ → Toll is aggregation: the toll booth physically exists regardless of any particular route or tour. Multiple Route records can reference the same Toll record.
- Future extension: cost fields could carry effective-date ranges if toll rates change seasonally, without altering the rest of the model.

5.8 REST API as Sole Communication Channel

All JavaFX UI interactions go exclusively through Spring Boot REST endpoints (HTTP/HTTPS). This enforces a clean separation of concerns — the UI layer contains no business logic — and opens the door to adding a web or mobile frontend in a future phase without changing the backend.

End of Document · Aktour ViaBalkan · CS308 · IUS 2026